



Chess Piece Recognition: Custom CNN vs YOLO

Honi Arora
Computer Vision & Deep Learning
Sapienza University of Rome



Table of contents

1

Problem & experimental design

2

Data pipeline

3

Models

4

Training & hyperparameter tuning

5

Results & analysis

6

Conclusions

Problem & experimental design

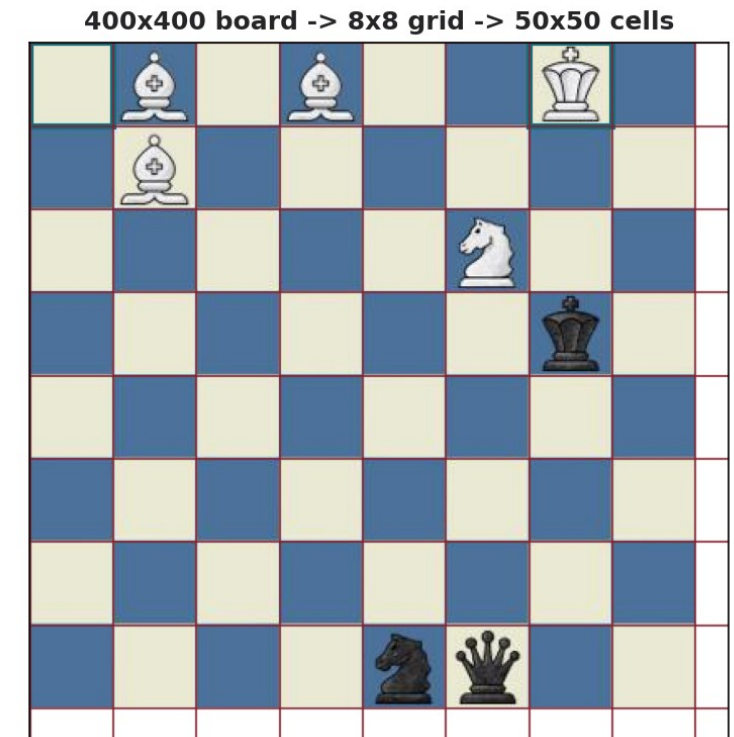
- Goal: recognize chess pieces from a board image; compare a small custom CNN classifier against YOLO detectors on accuracy, latency and compute.
- One shared synthetic dataset, processed into two views so both families see identical data:

View	Consumer	Unit	Labels
50×50 cell crops	Custom CNN	one board cell	13 (empty + 12 pieces)
cell bounding boxes	YOLO (pico/n/s)	full board image	12 piece classes

- Fair metric — “occupied-cell accuracy”: each YOLO detection is mapped to its grid cell; the top-confidence class per occupied cell is scored, putting detection on the CNN’s scale.

Data pipeline

- Source: koryakin/chess-positions (Kaggle, CC0) — 100k rendered 400×400 boards; label = FEN in the filename; board fills the frame as a clean 8×8 grid (50×50 cells).
- FEN → 8×8 grid → 13-class index per cell (empty=0, white 1-6, black 7-12); occupied cells → YOLO cell-boxes (12-class).
- Working set: 20k train / 4k test boards.
- Class skew: 83% empty, kings 1/board, queens ~0.8%; balanced to ~429k crops (max_empty_ratio).
- Memmap crop cache: precompute (N,64,50,50,3) uint8 → training reads crops with zero JPEG re-decode.
- Kornia differentiable, GPU-batched augmentation (affine/jitter/noise/blur), strength is a tuned hyperparameter.



Models



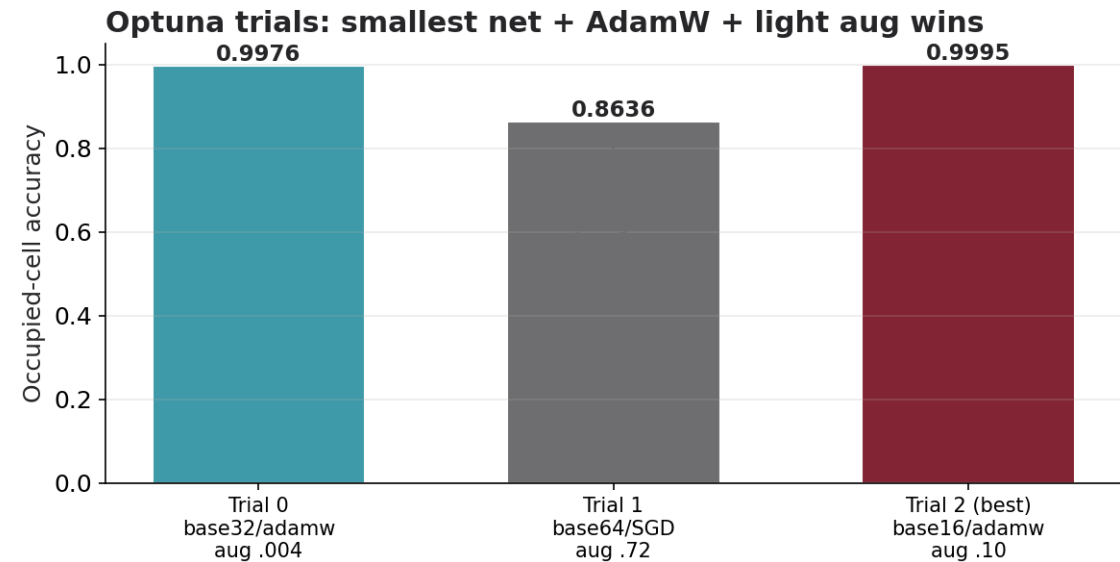
28,813 parameters · 0.0069 GFLOPs/cell · 125 KB on disk

YOLOv8 variants (trained on the same derived boxes):

Variant	Origin	Params	Note
yolo_pico	scaled YAML, from scratch	184,300	sub-nano stress test
yolov8n	pretrained (COCO)	2,692,548	standard nano
yolov8s	pretrained (COCO)	9,843,604	“big” model

Hyperparameter tuning (CNN)

- Optuna study, SQLite storage (resumable), MedianPruner; each trial → a wandb run.
- **Search space:**
 - lr, weight_decay (log); optimizer {adam/adamw/sgd}
 - aug_strength 0-0.8; n_blocks 2-4; base_ch {16/32/64}
 - fc_dim {64/128/256}; dropout 0-0.5; batch {128/256/512}
- Dual stop: target accuracy OR time budget; ETA predicted from a calibration mini-run.



- Insights: AdamW $\gg$ SGD; heavy augmentation hurts clean renders; the smallest net (28.8k params) won.



Training methodology

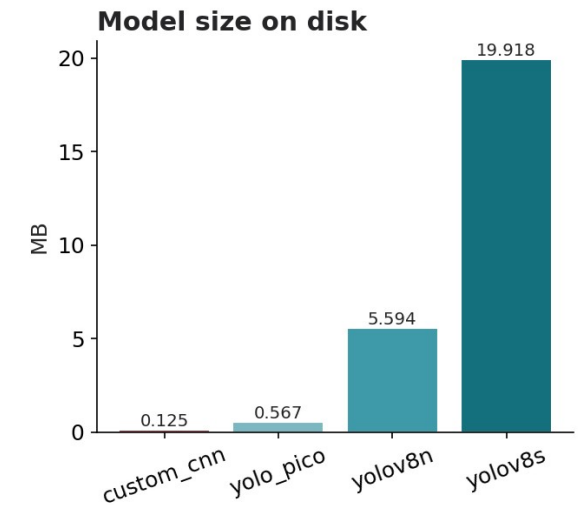
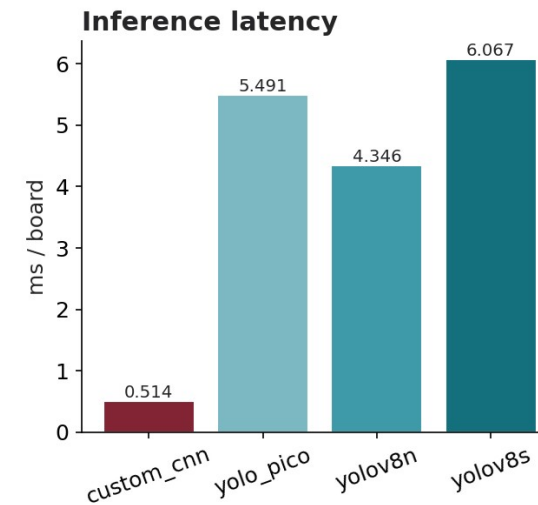
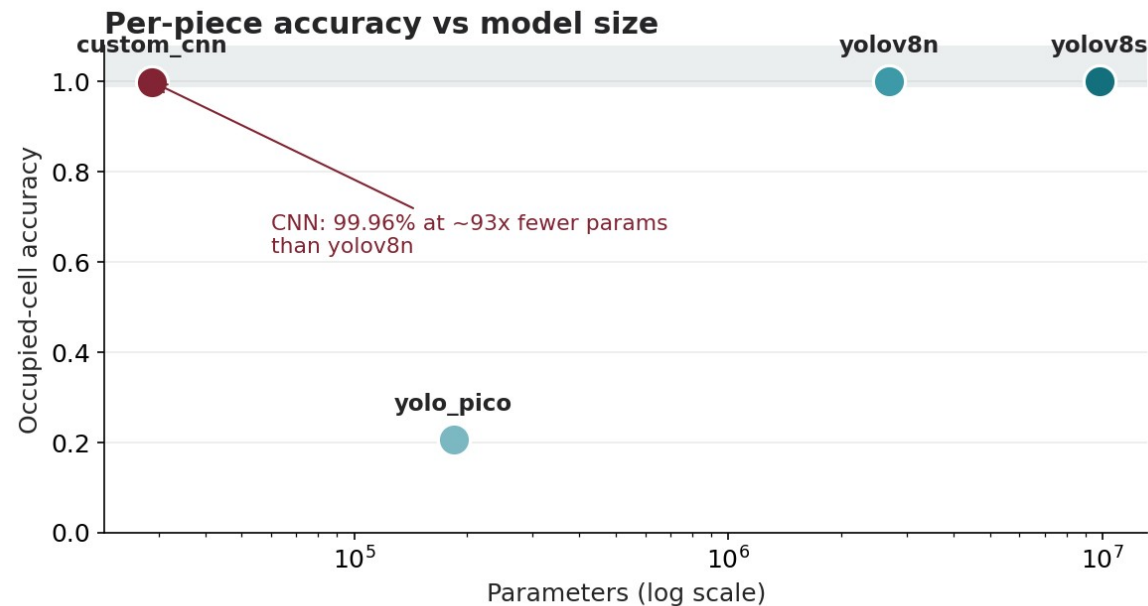
- Engineering: spec → plan → 19-task TDD build; per-task spec + code-quality review; 38 unit/smoke tests.
- Hardware-aware: RTX A4000 16 GB; 90% GPU-memory cap; monitored soft RAM guard (RLIMIT_AS breaks CUDA).
- CNN speed (measured): 0.0283 s/step → ~38 s/epoch → ~142 s per 6-epoch trial; target hit in 1 epoch.
- Two-phase tuning: target 0.995 was met instantly (trial 0) → raised to 0.999 to force real exploration.
- YOLO: imgsz 416, 12 epochs, 25% data fraction; n/s from COCO weights, pico from scratch; optimizer=auto (AdamW).
- Final CNN rebuilt from the best Optuna trial (FixedTrial), retrained, saved, pushed to Hugging Face.

Results — head-to-head

Model	Occ-acc	mAP@50-95	Params	GFLOPs/bd	Lat ms/bd	Disk MB	Train s
custom_cnn ★	0.9996	—	28,813	0.442	0.514	0.125	~38
yolo_pico	0.2073	0.255	184,300	0.236	5.491	0.567	260
yolov8n	1.0000	0.995	2,692,548	1.469	4.346	5.594	537
yolov8s	1.0000	0.995	9,843,604	4.985	6.067	19.918	734

- Pretrained YOLOs are perfect (grid-aligned boxes are trivial to localize); CNN trails by only 0.04 pp.
- CNN vs yolov8n: ~93× fewer params, ~8.5× lower latency, ~45× smaller — at ~equal accuracy.
- Pico (sub-nano, from scratch) collapses to 20.7% and is even slower than nano.

Results — accuracy vs cost



- Shrink the paradigm, not just the width: a tiny CNN classifier learns in 1 epoch; a tiny detector cannot.



Conclusions & recommendations

- On grid-croppable / embedded targets: ship the custom CNN — 99.96% at 125 KB and 0.5 ms/board.
- For robust real-photo recognition: yolov8n pretrained — perfect here, no board-cropping step, best generalization; yolov8s adds cost without accuracy.
- Do not train a sub-nano YOLO from scratch on small data.
- Trade-off: the CNN's efficiency assumes an 8×8 grid (free from synthetic geometry); real boards need detection/perspective first — YOLO's strength.
- Future work: real-photo test set (ChessReD2K), full-data YOLO + HPO sweep, board→FEN reconstruction.



Artifacts & reproducibility

- Code: github.com/Honi05/cvproject (TDD, 38 tests, PR #1).
- CNN model: huggingface.co/honi05/chess-piece-cnn
- YOLO models (pico/n/s): huggingface.co/honi05/chess-piece-yolo
- Derived dataset: huggingface.co/datasets/honi05/chess-positions-cv
- Experiment tracking: Weights & Biases — project chess-cnn-vs-yolo (Optuna trials + comparison).
- One command per stage: 01_build_data → 02_train_cnn → train_all_yolo → full_compare → push_artifacts.



References

- **Dataset: P. Koryakin, “Chess Positions”, Kaggle (CC0).**
 - Jocher et al., Ultralytics YOLOv8.
 - Akiba et al., “Optuna: A Next-generation Hyperparameter Optimization Framework”, KDD 2019.
 - Riba et al., “Kornia: an Open Source Differentiable CV Library for PyTorch”, WACV 2020.
 - Slides template: github.com/pietro-nardelli/sapienza-ppt-template (CC BY-NC-SA 4.0).



Thank you for the attention!